GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: SIV-SACHA

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	Núcleo transaccional (MVP) del E-commerce SIV-SACHA.
Operaciones MVP	Buscar producto, Registrarse, Añadir carrito y Realizar pedido
Referencia base	01_Guia_PlantUML_CU_Analisis_SIV-SACHA_G4
Propósito	Guiar el paso del análisis funcional al análisis estructural y dinámico, manteniendo coherencia entre requisito, caso de uso, clase y secuencia.

1. Propósito de la guía

Esta guía complementa la fase inicial. Su finalidad es mostrar cómo el núcleo transaccional de SIV-SACHA se analiza de forma progresiva mediante tres modelos: casos de uso, clases de análisis y secuencias. El énfasis es comprender la responsabilidad de cada elemento (Frontera, Control, Repositorio, Entidad) conservando la trazabilidad desde el requisito funcional hasta la interacción.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos que aparecerán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por operación del CRUD del estudiante	Valida que el flujo sea coherente con los casos de uso y las clases identificadas.

3. Caso base actualizado: Información central del SIV-SACHA

El caso se basa en una interfaz sencilla denominada CRUD Estudiantes, con tres campos de entrada y cuatro operaciones principales.

Dato	Tipo sugerido	Descripción	Validación mínima
Producto	Entidad / Objeto	Catálogo con ID, Nombre, Descripción, Precio, Stock e Imágenes.	ID único, precio y stock > 0.
Cliente	Entidad / Objeto	Datos de cuenta: ID/Cédula, Nombres, Correo, Clave y Dirección.	Correo único y formato válido.
Carrito	Entidad / Objeto (Temporal)	Colección temporal de los ítems seleccionados, cantidades y subtotal acumulado.	Cantidad solicitada ≤ stock.

Pedido	Entidad / Objeto	Registro de la transacción con detalle de ítems, total y dirección.	Carrito no vacío y dirección válida.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Buscar	Buscar producto	Buscar por nombre o categoría en la barra de búsqueda.
Crear Cuenta	Registrar	Registra al cliente validando nombre, correo y contraseña
Añadir al carrito	Añadir al carrito	Guarda temporalmente el ID del producto, cantidad y precio subtotal para la compra
Confirmar compra	Realizar Pedido	Valida el número de cédula y celular de contacto y se confirma el pedido.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir buscar productos en el catálogo.	Visitante - Cliente	Buscar Producto
RF-03	El sistema debe permitir la creación de cuentas.	Visitante	Registrarse
RF-05	El sistema debe permitir agregar productos al carrito.	Cliente	Añadir carrito
RF-06	El sistema debe permitir procesar la orden y confirmar.	Cliente	Realizar Pedido

5. Diagrama de casos de uso de análisis

El diagrama de casos de uso representa las funciones visibles para el actor. En este caso, ell Visitante será un usuario No Autenticado, el cual podrá tener opciones de visualizar información, como lo es el de Buscar Producto, además puede Registrarse, convirtiéndose en Cliente. El Cliente es quien puede realizar pedidos y Añadir al carrito, teniendo en cuenta que de igual manera que el Visitante, puede visualizar información, en este caso Buscar Producto.

Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Visitante" as Vis
actor "Cliente" as Cli

rectangle "SIV-SACHA" {
  usecase "Buscar producto" as UC1
  usecase "Registrarse" as UC2
  usecase "Añadir al carrito" as UC3
  usecase "Realizar pedido" as UC4
}

Vis --> UC1
Vis --> UC2

Cli --> UC1
Cli --> UC3
Cli --> UC4
@enduml
```

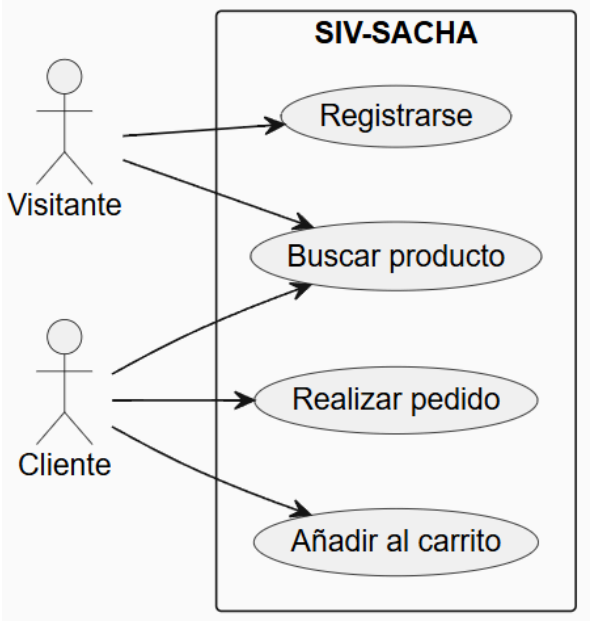


Figura 1. Casos de uso

6. Derivación de clases de análisis

A partir de los casos de uso se identifican clases de análisis. Para mantener una separación clara de responsabilidades se recomienda usar los estereotipos boundary, control y entity. Además, se puede incluir un repositorio para representar la persistencia o colección de estudiantes sin entrar aún en detalles técnicos de base de datos.

Tabla 6.1 - Derivación para CU-01: Buscar producto

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el Caso de Uso
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaCatalogo	Interfaz donde el visitante/cliente ingresa la categoría o término a buscar y visualiza la lista resultante.
Control	Coordina la lógica de cada caso de uso.	ControlBusqueda	Recibe el término ingresado, valida el texto y delega la instrucción de búsqueda.
Entity / Entidad	Representa la información del dominio.	Producto	Define los atributos (nombre, precio, imagen) de los resultados que se van a mostrar.
Repositorio / Colección	Conserva y recupera registros.	RepositorioProducto	Realiza la consulta directa a la base de datos para recuperar los productos que coinciden con la búsqueda.

Tabla 6.2 - Derivación para CU-02: Registrarse

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el Caso de Uso
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaRegistro	Formulario donde el visitante ingresa sus datos personales (nombre, correo, clave) para crear su cuenta.
Control	Coordina la lógica de cada caso de uso.	ControlRegistro	Valida el formato de los datos y coordina el proceso para asegurar que el usuario sea válido.

Entity / Entidad	Representa la información del dominio.	Cliente	Molde que estructura los atributos del nuevo usuario registrado.
Repositorio / Colección	Conserva y recupera registros.	RepositorioCliente	Verifica en la base de datos que el correo no esté duplicado y guarda permanentemente el nuevo registro.

Tabla 6.2 - Derivación para CU-03: Añadir Carrito

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el Caso de Uso
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaDetalleProd	Interfaz gráfica del producto donde el cliente selecciona la cantidad deseada y pulsa el botón de añadir.
Control	Coordina la lógica de cada caso de uso.	ControlCarrito	Ejecuta la regla de negocio verificando que la cantidad solicitada no exceda el inventario.
Entity / Entidad	Representa la información del dominio.	Carrito Producto	El carrito mantiene el estado temporal de los ítems en sesión, y el producto aporta el precio para calcular el subtotal.
Repositorio / Colección	Conserva y recupera registros.	RepositorioProducto	Es consultado por el control únicamente para verificar las unidades existentes de stock antes de permitir agregarlo al carrito.

Tabla 6.2 - Derivación para CU-04: Realizar Pedido

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el Caso de Uso
Boundary / Frontera	Recibe datos del actor y muestra resultados.	PantallaCheckout	Pantalla de pago donde el cliente visualiza el total, ingresa su dirección de envío y confirma la orden.
Control	Coordina la lógica de cada caso de uso.	ControlPedido	Procesa la transacción final, coordina el cierre del carrito y genera el comprobante de compra.
Entity / Entidad	Representa la información del dominio.	Producto / Pedido	Estructura el registro final con los detalles de la venta (ítems, total, fecha y cliente) para la facturación.
Repositorio / Colección	Conserva y recupera registros.	RepositorioPedido RepositorioProducto	**RepositorioProducto** Actualiza el inventario. **RepositorioPedido** Persiste la nueva orden en la base de datos..

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

' 1. CLASES FRONTERA (BOUNDARY)
class "PantallaCatalogo" <<boundary>> {
  - txtBusqueda: String
  + clickBuscar()
  + mostrarProductos(productos: List<Producto>)
}

class "PantallaRegistro" <<boundary>> {
  - txtNombre: String
  - txtCorreo: String
  - txtClave: String
```

```
  + clickRegistrar()
  + mostrarMensaje(mensaje: String)
}

class "PantallaDetalleProd" <<boundary>> {
  - txtCantidad: int
  + clickAnadirCarrito()
  + mostrarConfirmacion()
}

class "PantallaCheckout" <<boundary>> {
  - txtDireccion: String
  + clickPagar()
  + mostrarComprobante(pedido: Pedido)
}

' 2. CLASES CONTROL (CONTROL)
class "ControlBusqueda" <<control>> {
  + buscarProducto(categoria: String): List<Producto>
}

class "ControlRegistro" <<control>> {
  + registrarCliente(nombre: String, correo: String, clave: String): boolean
}

class "ControlCarrito" <<control>> {
  + agregarAlCarrito(idProducto: String, cantidad: int): boolean
}

class "ControlPedido" <<control>> {
  + procesarPedido(carrito: Carrito, direccion: String): Pedido
}

' 3. REPOSITORIOS (REPOSITORY)
class "RepositorioProducto" <<repository>> {
  + consultarPorCategoria(categoria: String): List<Producto>
  + verificarStock(idProducto: String, cantidad: int): boolean
  + descontarStock(carrito: Carrito): void
}

class "RepositorioCliente" <<repository>> {
  + existeCorreo(correo: String): boolean
  + guardar(cliente: Cliente): void
}

class "RepositorioPedido" <<repository>> {
  + guardar(pedido: Pedido): void
}

' 4. ENTIDADES (ENTITY)
class "Producto" <<entity>> {
  - id: String
  - nombre: String
  - precio: double
  - stock: int
  + Producto(id: String, nombre: String, precio: double, stock: int)
  + getId(): String
  + getPrecio(): double
  + getStock(): int
}

class "Cliente" <<entity>> {
  - id: String
  - nombre: String
  - correo: String
  - clave: String
  + Cliente(nombre: String, correo: String, clave: String)
  + getCorreo(): String
}

class "Carrito" <<entity>> {
  - subtotal: double
  + actualizarItems(idProducto: String, cantidad: int): void
  + getSubtotal(): double
}

class "Pedido" <<entity>> {
```

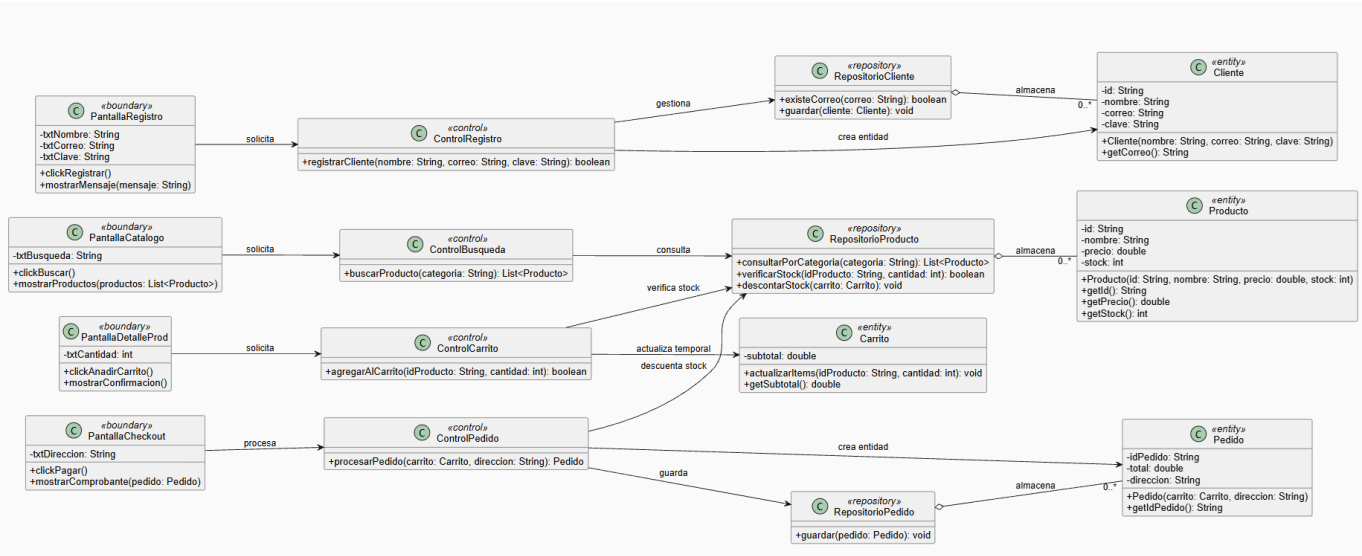
```
  - idPedido: String
  - total: double
  - direccion: String
  + Pedido(carrito: Carrito, direccion: String)
  + getIdPedido(): String
}

' RELACIONES ENTRE CLASES
PantallaCatalogo --> ControlBusqueda : solicita
ControlBusqueda --> RepositorioProducto : consulta
RepositorioProducto o-- "0..*" Producto : almacena

PantallaRegistro --> ControlRegistro : solicita
ControlRegistro --> RepositorioCliente : gestiona
ControlRegistro --> Cliente : crea entidad
RepositorioCliente o-- "0..*" Cliente : almacena

PantallaDetalleProd --> ControlCarrito : solicita
ControlCarrito --> RepositorioProducto : verifica stock
ControlCarrito --> Carrito : actualiza temporal

PantallaCheckout --> ControlPedido : procesa
ControlPedido --> RepositorioProducto : descuenta stock
ControlPedido --> RepositorioPedido : guarda
ControlPedido --> Pedido : crea entidad
RepositorioPedido o-- "0..*" Pedido : almacena
@enduml
```



7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Buscar producto	Recibe datos del actor y muestra resultados.	Coordina la lógica de cada caso de uso.	RepositorioProducto, Producto	Ingresar término -> consultar Repo -> devolver lista.
Registrarse	Recibe datos del actor y muestra resultados.	Coordina la lógica de cada caso de uso.	RepositorioCliente, Cliente	Ingresar datos -> validar correo -> crear Cliente -> guardar.
Añadir Carrito	Recibe datos del actor y muestra resultados.	Coordina la lógica de cada caso de uso.	RepositorioProducto, Carrito	Añadir cantidad -> validar stock en Repo -> actualizar Carrito.

Realizar Pedido	Recibe datos del actor y muestra resultados.	Coordina la lógica de cada caso de uso.	RepositorioProducto, RepoPedido, Pedido	Confirmar compra -> descontar stock -> generar Pedido -> guardar.

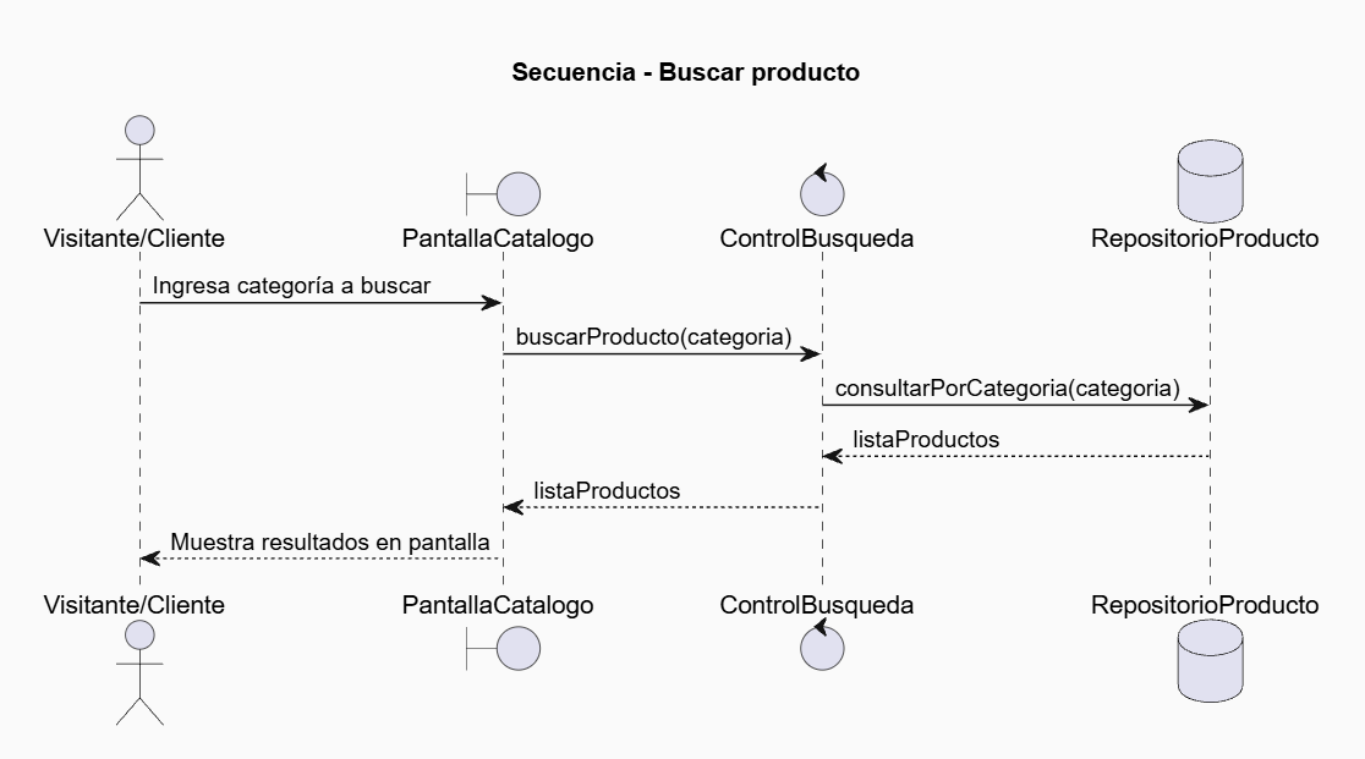
8. Diagramas de secuencia de análisis

Los diagramas de secuencia muestran cómo colaboran los objetos de análisis en el tiempo. Se recomienda elaborar un diagrama por cada caso de uso principal para evitar diagramas sobrecargados.

8.1 Secuencia: Buscar producto

```
@startuml
title Secuencia - Buscar producto
actor "Visitante/Cliente" as Actor
boundary "PantallaCatalogo" as UI
control "ControlBusqueda" as Ctrl
database "RepositorioProducto" as Repo

Actor -> UI : Ingresa categoría a buscar
UI -> Ctrl : buscarProducto(categoria)
Ctrl -> Repo : consultarPorCategoria(categoria)
Repo --> Ctrl : listaProductos
Ctrl --> UI : listaProductos
UI --> Actor : Muestra resultados en pantalla
@enduml
```

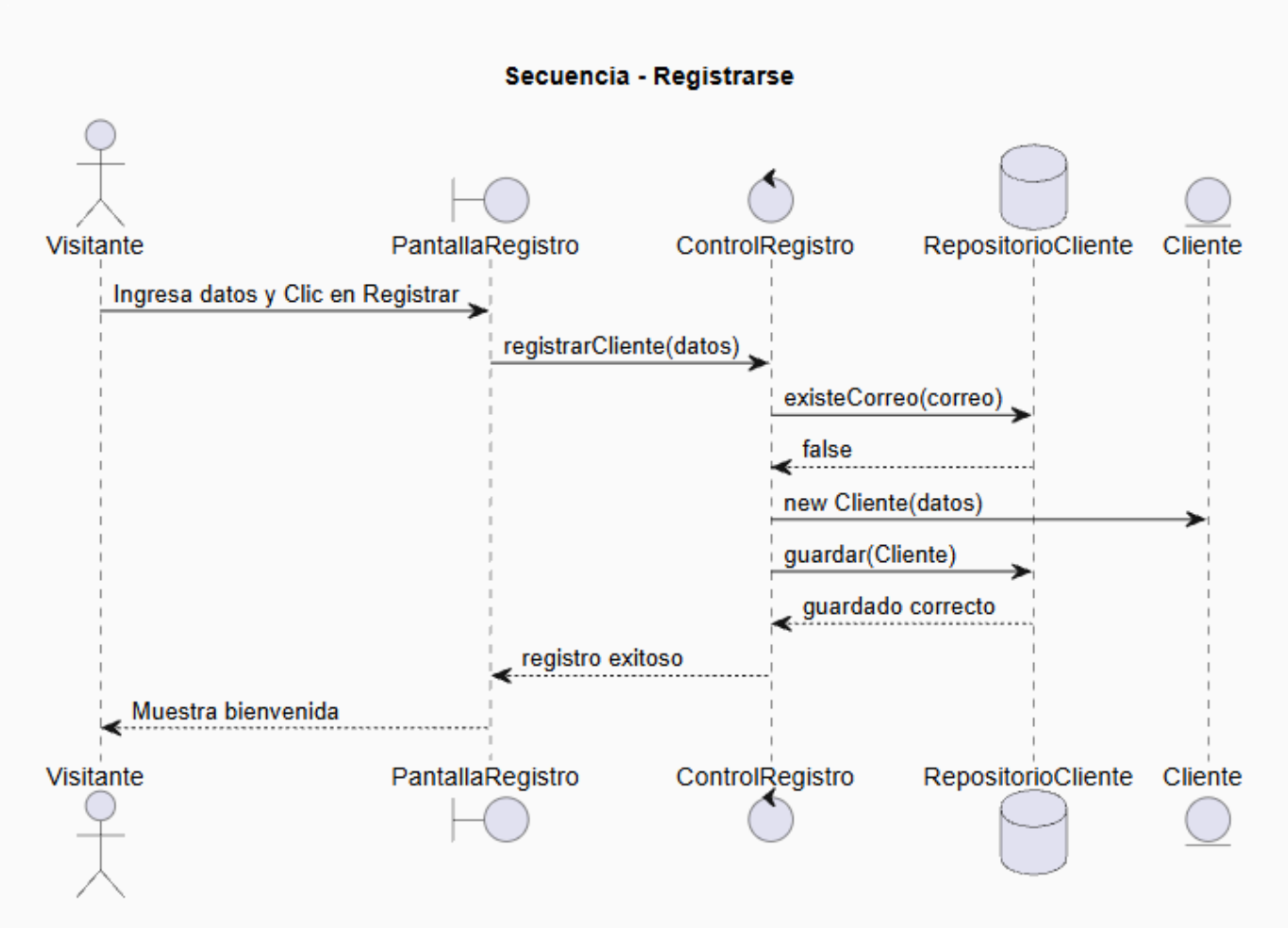


8.2 Secuencia: Registrarse

```
@startuml
title Secuencia - Registrarse
actor "Visitante" as Actor
boundary "PantallaRegistro" as UI
control "ControlRegistro" as Ctrl
database "RepositorioCliente" as Repo
entity "Cliente" as Ent

Actor -> UI : Ingresa datos y Clic en Registrar
UI -> Ctrl : registrarCliente(datos)
Ctrl -> Repo : existeCorreo(correo)
```

```
Repo --> Ctrl : false
Ctrl -> Ent : new Cliente(datos)
Ctrl -> Repo : guardar(Cliente)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : registro exitoso
UI --> Actor : Muestra bienvenida
@enduml
```



8.3 Secuencia: Añadir al carrito

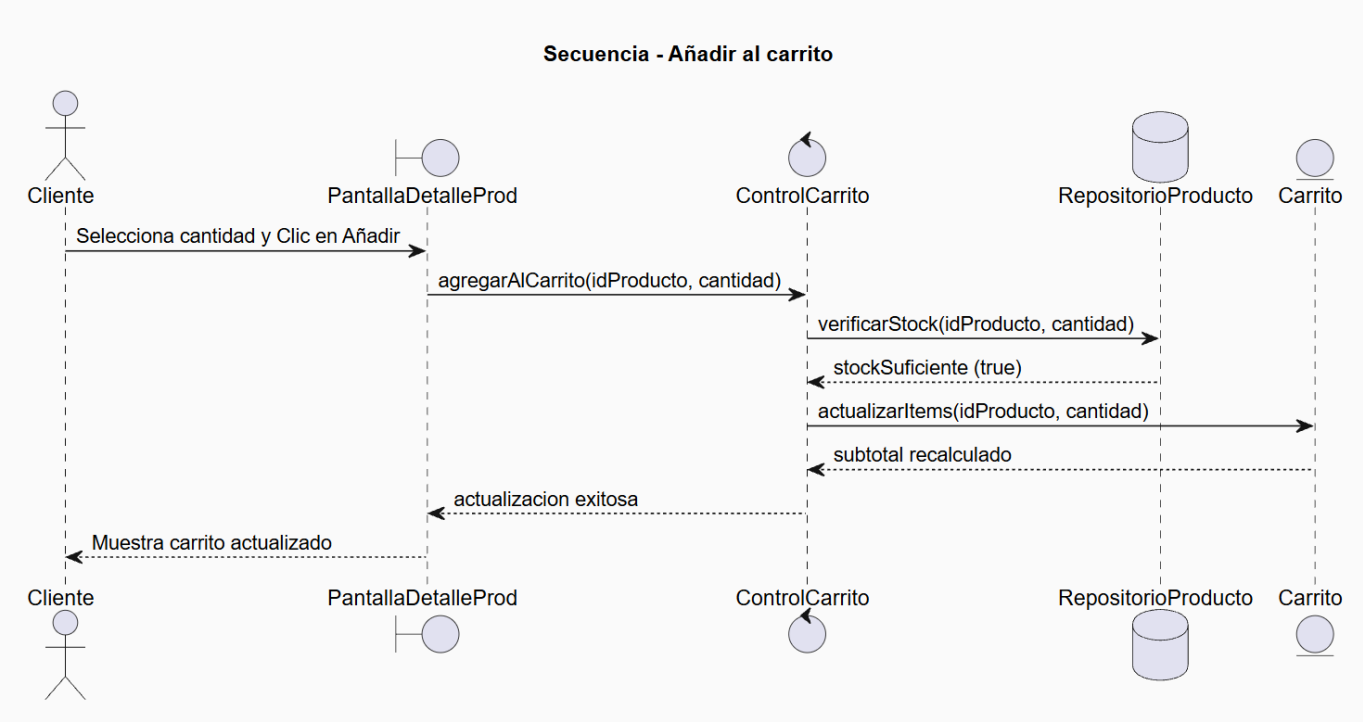
```
@startuml
title Secuencia - Añadir al carrito
actor "Cliente" as Actor
boundary "PantallaDetalleProd" as UI
control "ControlCarrito" as Ctrl
database "RepositorioProducto" as Repo
entity "Carrito" as Ent

Actor -> UI : Selecciona cantidad y Clic en Añadir
UI -> Ctrl : agregarAlCarrito(idProducto, cantidad)
Ctrl -> Repo : verificarStock(idProducto, cantidad)
Repo --> Ctrl : stockSuficiente (true)
Ctrl -> Ent : actualizarItems(idProducto, cantidad)
Ent --> Ctrl : subtotal recalculado
Ctrl --> UI : actualizacion exitosa
UI --> Actor : Muestra carrito actualizado
@enduml
```

8.4 Secuencia: Realizar pedido

```
@startuml
title Secuencia - Realizar pedido
actor "Cliente" as Actor
boundary "PantallaCheckout" as UI
control "ControlPedido" as Ctrl
database "RepositorioProducto" as RepoProd
database "RepositorioPedido" as RepoPed
entity "Pedido" as Ent

Actor -> UI : Confirma dirección y Clic en Pagar
UI -> Ctrl : procesarPedido(carrito, direccion)
Ctrl -> RepoProd : descontarStock(carrito)
RepoProd --> Ctrl : inventario actualizado
Ctrl -> Ent : new Pedido(carrito, direccion)
Ctrl -> RepoPed : guardar(Pedido)
RepoPed --> Ctrl : compra registrada
Ctrl --> UI : comprobante de pago
UI --> Actor : Muestra confirmación de compra
@enduml
```



9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Buscar Producto	Acceso a la plataforma web.	Ingresar filtro → consultar DB → retornar productos.	Tabla de productos visible.
Registrar	El correo no debe existir previamente.	Llenar formulario → validar unicidad → instanciar y guardar	Cliente agregado y sesión lista.
Añadir Carrito	El producto seleccionado existe.	Elegir cantidad → consultar stock → actualizar sesión.	Carrito actualizado con nuevo subtotal.
Realizar Pedido	Carrito con productos.	Confirmar envío → descontar inventario → guardar transacción	Pedido facturado y stock reducido.

10. Procedimiento recomendado para el estudiante

- Revisar el flujo transaccional del e-commerce SIV-SACHA e identificar los bloques de datos: Reconocer las entidades centrales del sistema: Producto (catálogo), Cliente (cuenta de usuario), Carrito (sesión temporal) y Pedido (transacción de compra).
- Redactar los requisitos funcionales de nivel 0 para el MVP: Definir los requisitos clave asociados al núcleo del negocio: Buscar producto (RF-01), Registrarse (RF-02), Añadir al carrito (RF-03) y Realizar pedido (RF-04).
- Modelar los casos de uso en PlantUML y justificar únicamente las relaciones include necesarias.
- Derivar clases de análisis: una clase frontera para la interfaz, una clase control para coordinar la lógica, una entidad Estudiante y un repositorio o colección.
- Construir un diagrama de secuencia por cada caso de uso principal.
- Verificar que cada mensaje del diagrama de secuencia corresponda a una operación declarada en el diagrama de clases.
- Actualizar la matriz de trazabilidad para demostrar coherencia entre RF, CU, clases y secuencias.

11. Lista de verificación de coherencia

- Cada requisito funcional tiene al menos un caso de uso asociado.
- Cada caso de uso principal posee una secuencia de análisis propia.
- Las clases usadas en las secuencias existen en el diagrama de clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de las clases de análisis.
- La validación de datos se aplica antes de agregar y actualizar.
- El diagrama no mezcla detalles de implementación de interfaz gráfica con responsabilidades de análisis.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Relaciona correctamente RF, casos de uso, clases y secuencias.	5 pts	Matriz de relación completa.
Diagrama de casos de uso	Representa actor, sistema, casos de uso y relaciones include justificadas.	4 pts	Código y diagrama PlantUML.
Diagrama de clases de análisis	Identifica frontera, control, entidad y repositorio con responsabilidades claras.	4 pts	Código PlantUML de clases.
Diagramas de secuencia	Modela flujos coherentes para Agregar, Actualizar, Eliminar y Mostrar Todo.	5 pts	Cuatro secuencias PlantUML.
Presentación técnica	Documento ordenado, lenguaje claro y sin errores de sintaxis PlantUML.	2 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Casos de uso	Requisitos funcionales, diagrama CU y especificación breve.	Código PlantUML + captura.
Clases de análisis	Diagrama con boundary, control, entity y repositorio.	Código PlantUML + captura.
Secuencias de análisis	Una secuencia por cada operación CRUD principal.	Códigos PlantUML + capturas.
Trazabilidad	Matriz que conecte RF, CU, clases y secuencias.	Tabla de relación.
Conclusión técnica	Reflexión breve sobre la coherencia entre modelos.	Texto de cierre.

14. Conclusión técnica

La relación entre casos de uso, clases de análisis y diagramas de secuencia permite comprobar que el modelo no se construye como piezas aisladas. En el Sistema Web SIV-SACHA, los casos de uso explican qué necesita el actor en el núcleo transaccional; las clases de análisis explican qué objetos colaboran organizados en frontera, control, repositorio y entidad; y las secuencias muestran cómo esos objetos interactúan paso a paso para cumplir cada operación. Esta trazabilidad garantiza la coherencia en el Análisis de Software, preparando de forma clara la base para la posterior implementación del sistema.

15. Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_CRUD_Estudiante.
- Guía Version2 del CRUD Estudiante con campos ID, Nombre y Edad, adaptada a la interfaz de referencia proporcionada por la docente.